04 — Runbook: From Zero to Final Submission

This is the operational checklist. It assumes:

- a remote Linux box with **4×A100 40 GB**, a recent NVIDIA driver/CUDA, and enough disk;
- a Kaggle account with competition rules accepted;
- the competition dataset (22.42 GB) and wheelhouse available;
- the public prompt-only bundle (author-reported 0.24) as the baseline (S36).

Commands use placeholders:

```
export WORK=~/gemma4
export DATA=$WORK/data
export RUNS=$WORK/runs
export AGENT=$WORK/submission
export HF_HOME=$WORK/hf
mkdir -p "$WORK" "$DATA" "$RUNS" "$WORK/agents" "$AGENT"
```

Do not run a command you do not understand. Every step has an expected result; stop if it doesn't match.

---

0. Preflight

0.1 Local workstation

The local workstation has no NVIDIA GPU, 15 GB RAM, and 57 GB free disk. Do not attempt 31B training locally. Use it for editing, reading, and packaging.

0.2 Remote box

```
nvidia-smi
free -h
```

```
df -h
python --version
pip --version
git --version
nvcc --version 2>/dev/null || true
```

Expected:

- 4×A100 40 GB visible.
- ≥64 GB RAM preferred; more if CPU offload.
- ≥250 GB free disk (models + data + checkpoints). [ESTIMATE]
- Python 3.10–3.12; a CUDA-compatible PyTorch.

Record the environment:

```
mkdir -p $WORK/env
{
  date -u
  nvidia-smi
  python --version
  pip freeze
  git --version
} > $WORK/env/preflight.txt 2>&1
```

0.3 Kaggle credentials

Create a Kaggle API token (Account → Settings → API → Create New Token). Put it at `~/.kaggle/kaggle.json` with mode 600:

```
mkdir -p ~/.kaggle
mv kaggle.json ~/.kaggle/kaggle.json
chmod 600 ~/.kaggle/kaggle.json
```

Install the CLI in a venv (not system Python):

```
python3 -m venv $WORK/venv
source $WORK/venv/bin/activate
pip install -U pip kaggle
kaggle competitions list --search gemma-4-developer-agent
```

Expected: the competition appears.

0.4 Day 0: join, accept rules, submit the probe

Do this before building any local infrastructure:

1. Open the competition page, click **Join**, and accept the rules. Confirm the submissions page says you have accepted (S3).
2. Accept the Gemma model license / authenticate for gated downloads:

   ```
   hf auth login
   hf download google/gemma-4-31B-it-qat-w4a16-ct --revision main --local-dir $WORK/models/gemma-4-31b-ct  # HF repo casing; competition alias is lowercase; [VERIFY] pin revision
   ```

   Record the model revision.
3. Package the public prompt-only files (S36) and upload `$WORK/submission.zip` as a probe. This costs one daily slot and zero final slots. Do not wait for local evaluation; the probe tests the submission mechanism.
4. Record the submission ref/status.

If the probe fails, stop and fix the submission mechanism before training. A second failed probe is worse than a mediocre score.

---

1. Get the competition data

1.1 Download

```
mkdir -p $DATA
cd $DATA
kaggle competitions download -c gemma-4-developer-agent -p $DATA --unzip
```

Expected: `tasks.jsonl`, `snapshots/`, `graphs/`, `embeddings/`, `wheels/`, `docker/`, `sandbox/`, `sample_submission/`, `HARNESS_README.md`.

Check:

```
du -sh $DATA
ls -la $DATA | head
```

```
python - <<'PY'
import json, pathlib
p=pathlib.Path("tasks.jsonl")
rows=[json.loads(l) for l in p.read_text().splitlines() if l.strip()]
print("tasks", len(rows))
print("repos", sorted({r["repo"] for r in rows}))
print("first", rows[0]["instance_id"], rows[0]["repo"])
PY
```

Expected: 129 tasks, four repos (S2). If the download is partial, re-run; do not train/eval on a partial dataset.

1.2 Wheelhouse

The starter notebook installs 41 wheels from:

```
/kaggle/input/datasets/metric/gemma-4-developer-agent-wheelhouse
```

On the remote box, download the wheelhouse dataset (or use the competition `wheels/` directory if it contains the harness wheels). The competition `wheels/` directory is for sandbox dependencies (S2); the harness wheelhouse is a separate Kaggle dataset (S6:55). Do not mix them.

```
# Example; adjust to the actual wheelhouse dataset path/name
kaggle datasets download -d metric/gemma-4-developer-agent-wheelhouse -p
$WORK/wheelhouse --unzip
ls $WORK/wheelhouse | head
```

Install the harness in a clean venv. The notebook does four preprocessing steps before `pip install --no-deps --force-reinstall` (S6:57-91): delete broken cutlass `.pth` hooks, skip cutlass wheels, restore the `cu128` → `+cu128` PEP 440 filename tag, and stage wheels in `/tmp/wheelhouse`. Reproduce them; a bare glob install will not match the notebook environment.

```
# 1) create the venv first; use system site-packages so --no-deps has the
image deps
python3 -m venv --system-site-packages $WORK/harness-venv
source $WORK/harness-venv/bin/activate

# 2) remove broken cutlass .pth hooks from the active venv
#    (on Kaggle, also check /usr/local/lib/python*/... as S6:58-66 does)
find "$VIRTUAL_ENV/lib" -maxdepth 3 -name '*cutlass*.pth' -delete
```

```
2>/dev/null || true

# 3) stage wheels: skip cutlass, restore +cu128
rm -rf /tmp/wheelhouse && mkdir -p /tmp/wheelhouse
for w in $WORK/wheelhouse/*.whl; do
  case "$(basename "$w")" in *cutlass*) continue;; esac
  name=$(basename "$w")
  case "$name" in *cu128*+*) ;; *cu128*) name=${name/cu128/+cu128};; esac
  ln -sf "$w" "/tmp/wheelhouse/$name" 2>/dev/null || cp "$w"
"/tmp/wheelhouse/$name"
done

# 4) install
pip install -U pip
pip install --no-deps --force-reinstall /tmp/wheelhouse/*.whl
python -c "import swegemma, adk_submission; print('harness ok')"
```

The Kaggle dataset slug `metric/gemma-4-developer-agent-wheelhouse` is inferred from the notebook's input path (S6:55). If the download 403s, discover the current slug with `kaggle datasets list -s gemma-4` (S41) and record the dataset version/hash.

If import fails, inspect the wheel names and versions; record them.

---

2. Build the prompt-only baseline

2.1 Create the submission directory

Use the public prompt-only files (S36; author-reported 0.24). Create:

```
$AGENT/
├── agent.yaml
├── configs/sampling.yaml
├── eval_config.yaml
└── prompts/system.md
```

`agent.yaml`:

```
name: swe_agent
model: gemma-4-31b-it-qat-w4a16-ct
instruction: !include prompts/system.md
tools:
  - run_command
  - read_file
```

```
    - edit_file
    - write_file
    - get_status
    - submit_patch
    - get_code_neighbors
    - search_similar_code
    - get_code_subgraph
  generate_content_config: !include configs/sampling.yaml
```

`configs/sampling.yaml`:

```
  temperature: 0.2
  top_p: 0.95
  max_output_tokens: 8192
  thinking_config:
    thinking_budget: 4096
    include_thoughts: true
```

`eval_config.yaml`:

```
  evaluation:
    timeout_seconds: 240
    max_tool_calls: 28
    max_time_minutes: 8
    max_turns: 80
```

`prompts/system.md`: use the public prompt-only prompt verbatim (S36; author-reported 0.24) or the shorter skeleton in `02-agent-engineering.md`. Keep it under ~4k characters.

Primary budget vs reproduction budget. The files above reproduce the public bundle: 8 min, 28 calls, 4096 thinking. For the primary 5-min bundle, change `eval_config.yaml` to `max_tool_calls: 24`, `max_time_minutes: 5`, `max_turns: 60`, change `thinking_budget` to `2048`, and update the prompt's hard-limit line from "28 tool calls and 8 minutes" to "24 tool calls and 5 minutes." Keep the 8-min zip as a known-good fallback.

Remove the sample dummy adapters. The provided `sample_submission` contains `adapters/main_lora` and `adapters/tool_lora` dummy adapters; `discover_adapters()` registers every adapter (S5:191-201, S6:246-247). Unless you trained one, delete `adapters/` **and** remove every `adapter:` reference from the YAML; otherwise validation/model discovery can point at a missing adapter. Assert

`/v1/models` lists zero adapters. Two dummy adapters reduce KV cache and can be silently applied.

2.2 Validate the directory

```
python - <<'PY'
import os
from pathlib import Path
from adk_submission import validate_directory
from swegemma.config import build_submission_limits
limits, _ = build_submission_limits()
validate_directory(Path(os.environ["AGENT"]), limits=limits)
print("submission directory valid")
PY
```

If this fails, fix the YAML/paths before going further.

2.3 Package

```
cd $AGENT
zip -r -X $WORK/submission-baseline.zip .
unzip -l $WORK/submission-baseline.zip
```

Expected: `agent.yaml` at the root, no `__MACOSX`, no symlinks. The starter notebook uses `root_dir=AGENT_DIR` (S6:600-601); this `cd` + `zip .` is equivalent. Zip entry order is not guaranteed, so verify with `unzip -l`, not by assuming the first path.

---

3. Serve the model locally

You need the model weights. On Kaggle, the model is mounted at:

```
/kaggle/input/models/google/gemma-4/other/<model>/2
```

On the remote box, download the official QAT W4A16 CT model from Hugging Face/Kaggle Models. Use the exact model name and revision.

3.1 Environment

```
export VLLM_MEMORY_PROFILER_ESTIMATE_CUDAGRAPHS=1
export VLLM_ENGINE_READY_TIMEOUT_S=1200
export TRANSFORMERS_OFFLINE=1
export VLLM_NO_USAGE_STATS=1
export VLLM_WORKER_MULTIPROC_METHOD=spawn
export PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
```

3.2 Serve

Activate the harness venv first (`source $WORK/harness-venv/bin/activate`) so `vllm`/`swegemma` are the same build. Use the harness's `VllmServer` or an equivalent command. The essential flags (S5:147-154, S6:272-290):

```
vllm serve "$WORK/models/gemma-4-31b-ct" \
  --served-model-name gemma-4-31b-it-qat-w4a16-ct \
  --host 127.0.0.1 --port 8000 \
  --tensor-parallel-size 4 \
  --max-model-len 32768 \
  --gpu-memory-utilization 0.90 \
  --enable-auto-tool-choice \
  --tool-call-parser gemma4 \
  --reasoning-parser gemma4
```

If using LoRA:

```
  --enable-lora --max-loras 1 --max-lora-rank 16 \
  --lora-modules main_lora=/path/to/adapters/main_lora
```

`max_loras`/`max_lora_rank` are derived from the adapter map (S5:153); for one rank-16 adapter the values above are correct, but do not hardcode them for a different rank.

Do not copy this command blindly if the harness's `VllmServer` adds flags; the scorer uses its own builder. The point is to match `max_model_len`, TP, parser, and memory settings. If you serve manually, register the endpoint/model alias with the harness (or use the starter notebook's `VllmServer`); otherwise `swegemma eval` may start its own server and ignore yours.

3.2b Build the sandbox image

`swegemma eval --sandbox docker` needs `swebench-sandbox:latest`. The competition ships Docker build files under `$DATA/docker/` (S2, S5:234). Build/load the image once, record the digest, and run a one-task smoke test before a full evaluation:

```
docker build -t swebench-sandbox:latest -f $DATA/docker/Dockerfile.sandbox
$DATA/docker
docker image inspect swebench-sandbox:latest --format '{{.Id}}' | tee
$WORK/env/sandbox-image.txt
```

If the Dockerfile context differs, follow the competition `docker/` instructions. Do not spend a week on a missing image; use the subprocess backend only as a fallback and record that it is not scorer-identical (S5:240).

3.3 Smoke test

```
curl -s http://127.0.0.1:8000/v1/models | head
```

Expected: base model (and adapter if enabled). Then run one public task through the harness:

```
swegemma eval \
  --tasks $DATA/tasks.jsonl \
  --snapshots-dir $DATA/snapshots \
  --submission-dir $AGENT \
  --results-dir $RUNS/smoke \
  --sandbox docker \
  --task-id fastapi_15661 \
  --max-tool-calls 24 \
  --max-time-minutes 5 \
  --concurrency 1 \
  --display auto
```

Expected: a non-empty patch or a clear reason (budget/timeout). If `NO_PATCH`, inspect the trace before blaming the model:

```
ls $RUNS/smoke
cat $RUNS/smoke/summary.json
```

```
cat $RUNS/smoke/traces/trace_fastapi_15661.json | head -80
```

Look for: first tool call, first edit, repeated calls, budget exhaustion, tool errors, context errors.

---

4. Calibrate local evaluation

A plain local notebook can fail gold patches (S31). Build a scorer-like environment or use a known calibrated sandbox.

4.1 Gold-patch check (10–15 tasks, not 127/129)

For 10–15 tasks spanning all four repos (at least 2 fastapi, 2 rich, 2 requests, 1 httpx):

1. Apply the task's reference `patch` to a fresh snapshot.
2. Apply `test_patch`.
3. Run the harness's Phase 2 command.
4. Confirm resolved.

Gate: **≥90% gold pass and 0 empty-patch resolves**. Mark the rest "uncalibrated" and never compare on them. A full 127/129 rebuild is a research artifact, not a week-2 requirement (S31, S32).

4.2 Baseline vs empty

Run `--skip-agent-patch` on a few tasks. The baseline should fail (or you should know why). If an empty patch resolves, the task/environment is broken for local comparison.

4.3 Holdout

Create a 20–40 task holdout, grouped by repository. Do not tune on all 129. Keep a separate set for final validation.

Record the task IDs:

```
python - <<'PY'
import json, random, os
rows=[json.loads(l) for l in open(os.path.join(os.environ["DATA"],
"tasks.jsonl")) if l.strip()]
random.seed(42)
# Example: hold out 33 tasks, stratified by repo
by={}
for r in rows: by.setdefault(r["repo"],[]).append(r["instance_id"])
hold=[]
for repo, ids in by.items():
    random.shuffle(ids)
    hold += ids[:max(1, round(len(ids)*0.25))]
print(len(hold))
open("holdout.txt","w").write("\n".join(hold))
print("\n".join(hold))
PY
```

Save the list; never train on these tasks.

---

5. Measure the baseline

Run the prompt-only baseline on the holdout:

```
swegemma eval \
  --tasks $DATA/tasks.jsonl \
  --snapshots-dir $DATA/snapshots \
  --submission-dir $AGENT \
  --results-dir $RUNS/baseline-$(date -u +%Y%m%d-%H%M) \
  --sandbox docker \
  --task-ids $(cat holdout.txt | tr '\n' ' ') \
  --max-tool-calls 24 \
  --max-time-minutes 5 \
  --concurrency 1 \
  --display auto
```

Collect:

```
python - <<'PY'
import json, pathlib, os
p=sorted(pathlib.Path(os.environ["RUNS"]).glob("baseline-*/summary.json"))
[-1]
s=json.loads(p.read_text())
print(json.dumps(s, indent=2)[:2000])
PY
```

Metrics:

- resolved count / total;
- patch rate (non-empty patches);
- `agent_p50`/`agent_p95` task time;
- `setup_p50`/`setup_p95` (container setup; the global 12h includes it; S5:511, S1);
- tool calls per task;
- malformed/repeated-call counts from traces;
- tasks hitting the time cap;
- projected `startup + 120 × (agent_p50 + setup_p50)`;
- A100→L4 derating: compare measured decode tok/s to the public 4×L4 ~33–35 tok/s for 31B (S34), or use the Kaggle L4x4 rehearsal.

If the baseline resolves <10% on the holdout, fix the scaffold before training. A LoRA cannot rescue an agent that never edits.

5.1 Kaggle L4x4 rehearsal (once, week 3)

Reserve one Kaggle L4x4 session for a full-stack rehearsal with the current candidate zip. L4x4 consumes GPU quota at 2× (S1, S42), so a 12h run costs about 24h of weekly quota; use it once, not for iteration. Record:

- wheelhouse hash and vLLM version;
- 31B decode tokens/s and per-call latency;
- `agent_p50`/`setup_p50` on a 10–15 task slice;
- projected 120-task time;
- any startup/queue errors.

This is the only hardware that matches the scorer. Use its timing as authoritative over A100 projections.

---

6. Generate trajectories

6.1 Run multiple temperatures

[ESTIMATE] 129 tasks × 5 temperatures ≈ 645 runs; at ~6 min wall each that is ~65 h sequential. With 8 concurrent requests against one 31B server it is ~8 h of wall time, if

the KV cache/headroom allows. If the week slips, use 2 temperatures (~258 runs); quality filtering matters more than raw count.

Use the same prompt/tools/thinking as inference. For each of five temperatures between 0.2 and 1.0 (e.g. 0.2, 0.4, 0.6, 0.8, 1.0), run the public tasks (or a large subset) and save results in separate directories:

```
for T in 0.2 0.4 0.6 0.8 1.0; do
  # copy the agent dir and edit temperature in configs/sampling.yaml
  cp -r $AGENT $WORK/agents/t$T
  sed -i "s/^temperature:.*/temperature: $T/"
$WORK/agents/t$T/configs/sampling.yaml
  swegemma eval \
    --tasks $DATA/tasks.jsonl \
    --snapshots-dir $DATA/snapshots \
    --submission-dir $WORK/agents/t$T \
    --results-dir $RUNS/gen-t$T \
    --sandbox docker --concurrency 2
done
```

Concurrency is for local throughput; it does not represent the scorer's global time. Keep the GPU server stable.

6.2 Keep resolved trajectories

For each run, parse `task_results.jsonl` and keep `resolved=True`. For each task, choose the shortest successful trajectory by tool-call count, or rank by quality.

6.3 Build per-call samples

For each trace:

1. Read the steps (ATIF: system/user/agent, tool calls, observations; S17).
2. For each assistant step, reconstruct the exact prompt messages and the assistant completion.
3. Render with the Gemma 4 chat template and the same tool schemas; verify token counts against the trace if available.
4. Tokenize; set labels `-100` for prompt/tool tokens; keep assistant tokens.
5. Save `input_ids`, `attention_mask`, `labels`, plus metadata (task, repo, step, model, revision, template hash).

If you cannot reconstruct token IDs reliably, instrument the serving path to log the exact prompt and output token IDs. Do not concatenate raw outputs; the runtime re-renders history (S39).

6.4 Filter

- resolved only;
- no test/config edits;
- no reads outside `/workspace`;
- no long identical-call streaks;
- no environment-failure tasks;
- thinking present if inference uses thinking;
- dedupe;
- hold out entire tasks/repos.

Write a dataset card:

```
source tasks, repo split, generation model/revision, prompt hash, tool
schema hash,
temperatures, filters, counts, token stats, license, known limitations.
```

Freeze the prompt and tool schema for the whole adapter line. Use the same `prompt_sha256` and `tool_schema_sha256` for generation, adapter evaluation, and submission. If you change either after generation, regenerate a 30-task calibration slice or drop the adapter (see `06-experiments-and-tracking.md`).

Publish the dataset if you use it (S3).

---

7. Train the adapter

7.1 Smoke test first

Do not launch a full run before a 10-step smoke test.

For Axolotl:

```
accelerate launch -m axolotl.cli.train config.yml --max_steps 10
```

`--max_steps` may not be a supported CLI override in every Axolotl version; if it is rejected, set `max_steps: 10` in the YAML for the smoke test. Check the current launcher docs. For Unsloth/TRL, set `max_steps=10`.

Check:

- loss is finite and decreasing-ish;
- peak VRAM per GPU;
- tokens/s;
- no NCCL/OOM errors;
- a checkpoint/adapter is saved.

Estimate:

```
tokens/s = global_batch_size * sequence_len * 10 / elapsed_seconds
epoch_time ≈ (num_samples / global_batch_size) / steps_per_second
```

7.2 Full run

Start with 1 epoch; evaluate; then 2–3 if underfitting. Keep the best checkpoint by validation loss or a small task eval, not training loss alone.

Monitor:

- grad norm (Gemma 4 can spike; `max_grad_norm: 0.1`, LR 5e-5; S11);
- loss (Gemma 4 31B loss is typically 1–3; S10);
- VRAM;
- tokens/s;
- checkpoint size.

7.3 Export

```
OUT=${1:-outputs/gemma4-31b-qlora}
ls "$OUT"/adapter_config.json "$OUT"/adapter_model.safetensors
du -sh "$OUT"
```

Copy only the adapter files into `$AGENT/adapters/main_lora/`.

Check keys/rank:

```
python - <<'PY'
import json
from safetensors.torch import load_file
p="$AGENT/adapters/main_lora"
c=json.load(open(f"{p}/adapter_config.json"))
sd=load_file(f"{p}/adapter_model.safetensors")
print({k:c.get(k) for k in
["r","lora_alpha","target_modules","base_model_name_or_path"]})
print("tensors", len(sd), "non-lora", [k for k in sd if "lora" not in k]
[:5])
PY
```

Add `adapter: main_lora` to `agent.yaml` (and any sub-agent that should use it).

---

8. Validate the adapter

8.1 Start vLLM with the adapter

Use the **same wheelhouse vLLM build** as the scorer/harness (the `$WORK/harness-venv` from §1.2), not an arbitrary local install. Record `vllm.__version__` and the wheel hash. Use the same env var and config as §3.2. Check:

```
curl -s http://127.0.0.1:8000/v1/models
```

Look for the adapter name and the base model. Check the server log for `GPU KV cache size` and any warnings.

8.2 A/B at temperature 0

Use three fixed prompts. Compare base vs adapter outputs. They must differ; if identical, the adapter is ignored (S23). Restart and repeat; outputs should be stable (S24).

8.3 Alpha sweep

Pin rank 16. Edit `adapter_config.json` `lora_alpha` to 8 or 16 (scale 0.5/1.0); 32 is a diagnostic only because scale 2.0 is the documented degenerate point at rank 16. Restart; run 5–10 tasks. Measure loops, time to first edit, patch rate, resolved. Pick the best trade-off.

8.4 Paired holdout

Run base and adapter on the same 20–40 holdout tasks, same seed, alternate order. Record:

- resolved;
- patch rate;
- p50/p95 time;
- calls;
- malformed/repeated calls.

Promote only if the adapter has ≥8 discordant pairs and ≥75% wins (or two seeds), does not regress p95/patch rate, and has a hidden score before final selection.

---

9. Package the final submission

9.1 Validate

```
python - <<'PY'
import os
from pathlib import Path
from adk_submission import validate_directory
from swegemma.config import build_submission_limits
agent=Path(os.environ["AGENT"])
for root, dirs, files in os.walk(agent, followlinks=False):
    for name in (*dirs, *files):
        p=Path(root)/name
        assert not p.is_symlink(), f"symlink: {p}"
limits, _ = build_submission_limits()
validate_directory(agent, limits=limits)
print("valid")
PY
```

9.2 Zip

```
cd "$AGENT"
zip -r -X "$WORK/submission.zip" .
unzip -l "$WORK/submission.zip"
```

Check:

- `agent.yaml` at root;
- `adapter_config.json` + `adapter_model.safetensors` if used;
- no `__MACOSX`, `.DS_Store`, caches, checkpoints, or optimizer states;
- unpacked size ≤3,221,225,472 bytes (the README says `<3 GiB`; stay below the constant);
- every non-directory entry has an allowed extension.

```
python - <<'PY'
import os, zipfile
from pathlib import Path
from swegemma.config import MAX_SUBMISSION_SIZE_BYTES,
ALLOWED_SUBMISSION_EXTENSIONS
zf=zipfile.ZipFile(os.environ["WORK"] + "/submission.zip")
total=sum(i.file_size for i in zf.infolist())
assert total <= MAX_SUBMISSION_SIZE_BYTES, f"too large: {total}"
for i in zf.infolist():
    if not i.is_dir():
        assert Path(i.filename).suffix.lower() in
ALLOWED_SUBMISSION_EXTENSIONS, i.filename
print("size/extensions ok:", total)
PY
```

9.3 Submit

Browser: Competition → Submit Prediction → File Upload → `$WORK/submission.zip` → description → Submit.

CLI:

```
kaggle competitions submit -c gemma-4-developer-agent -f
"$WORK/submission.zip" -m "prompt-only baseline v1"
```

Check status:

```
kaggle competitions submissions -c gemma-4-developer-agent -v
```

Do not submit multiple times in one day; the quota is one/day and a pending submission may block another (S3, S30).

---

10. Monitor and iterate

10.1 Queue

Recent queue delays have been ~14–33h (S30). A submission may take a day to score. Submit every 36–48h when you have a meaningful change; a 39-day window yields ~15–20 attempts, more than the 6–10 the plan needs. Do not hold a slot for an unvalidated idea.

10.2 Failure modes

- `Notebook Timeout` / global timeout: no partial score (S29). Reduce mean time or per-task caps.
- `Notebook Threw Exception`: often vLLM startup; check the forum and retry only if you have a slot (S30).
- No score / pending: wait; do not resubmit until the pending run finishes.
- Low score: compare to local, inspect variance; don’t overfit one run.

10.3 Keep a known-good bundle

Always keep the last scored prompt-only bundle. If an adapter experiment fails, you can resubmit the known-good bundle (subject to the daily quota).

10.4 Final selection

Up to two final submissions count (S3). Select:

1. the best validated 31B prompt-only or adapter bundle;
2. a diverse challenger (different prompt/budget/adapter) that has a hidden score.

Do not select two near-identical noisy variants. If you select fewer than two, Kaggle auto-selects from your best scoring submissions (S1).

---

11. Daily checklist

- ☐ GPU box healthy (`nvidia-smi`, disk, RAM).
- ☐ No runaway training/eval jobs.
- ☐ Today's run has a manifest (commit, config, prompt, adapter, data, seed).
- ☐ Local gold-patch check still passes.
- ☐ Submission quota/queue checked.
- ☐ Known-good bundle backed up.
- ☐ One variable changed since last experiment.
- ☐ Results logged in `docs/06-experiments-and-tracking.md` format.
- ☐ No test/config edits or scratch files in the patch.
- ☐ Final selection slots tracked.

---

12. Emergency procedures

12.1 Training OOM

- Reduce sequence length (8192 → 4096).
- Reduce micro-batch to 1; increase grad accumulation.
- Enable gradient checkpointing/activation offloading.
- Lower rank (16 → 8).
- Use QLoRA instead of BF16 LoRA, or vice versa.
- Move to 12B/E4B for a pipeline test.

12.2 Adapter startup OOM

- Ensure `VLLM_MEMORY_PROFILER_ESTIMATE_CUDAGRAPHS=1`.
- One adapter, rank 8/16; if rank 8, sweep alpha 4/8.
- Lower `gpu_memory_utilization` only if the scorer allows it; you cannot change the scorer's flags. Keep the adapter small.
- Test a 28k prompt; if it queues, the KV cache is too small.

12.3 Adapter ignored

- Check the vLLM version/build for the Gemma 4 LoRA fix (S23).
- Pin the base revision.
- Check target module names.
- A/B at temperature 0; if identical, do not submit the adapter.

12.4 Global timeout risk

- Reduce `max_time_minutes` (8 → 6 → 5).
- Reduce thinking (4096 → 2048/512).
- Reduce calls (28 → 24).
- Shorten the prompt's exploration phase; edit earlier.
- Measure p95, not mean.

12.5 Queue/score failure

- Wait for the pending run to finish.
- Check the forum for platform incidents.
- Keep the known-good bundle; resubmit when a slot is available.
- Do not burn the final days on infrastructure debugging.